



Correction de la Série d'Exercices TD/TP 05

Module : Algorithmique et Python 2

Filière : MIP

Semestre : S2

Etablissement : FSSM

Année Universitaire : 2023/2024

Corrigé par : Passerelle, Centre de Formation et d'Orientation

Adresse : LOT SIDI ABBAD 1 IMM N 298 ETG 3 BUR N 16

Email : info@passerelle.ma

Téléphone : +212 666 92 44 62 / +212 524 30 73 09

Introduction

Chers étudiants,

Ce document présente la correction de la série d'exercices TD/TP 05 du module "Algorithmique et Python 2" de la filière MIP, semestre S2, pour l'année universitaire 2023/2024, à la Faculté des Sciences Semlalia de Marrakech (FSSM).

L'objectif de cette série d'exercices est de vous familiariser avec divers algorithmes de tri et de recherche, qui sont des concepts fondamentaux en algorithmique. Les algorithmes étudiés incluent le tri à bulles (bubble sort), le tri par insertion (insertion sort), le tri par sélection (selection sort), ainsi que la recherche binaire (binary search).

Les corrections fournissent une explication détaillée de chaque étape de l'algorithme, avec un suivi précis des variables et des itérations. Cela vous permettra de mieux comprendre le fonctionnement interne de ces algorithmes et de renforcer vos compétences en programmation Python.

Nous espérons que ce document vous sera utile dans votre apprentissage et vous aidera à maîtriser les concepts abordés. N'hésitez pas à contacter votre professeur pour toute question ou clarification supplémentaire.

Bonne lecture et bon travail !

Professeur : ER-RAHMOUNY Hamza

Exercice 01 :

Pour chacun des algorithmes de tri suivants, suivez et notez le changement de la variable « arr »

Tri à Bulles :

L'algorithme de tri à bulles :

```
def tri_bulles(arr):
    n = len(arr)

    for i in range(n):
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier
        for j in range(0, n - i - 1):
            # Parcourez le tableau de 0 à n-i-1
            # Échangez si l'élément trouvé est supérieur à l'élément suivant
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr

a = [12, 11, 13, 5, 6]
a_trié = tri_bulles(a)
a_trié
```

[5, 6, 11, 12, 13]

Le Suivi de l'algorithme :

Pour le tableau [12, 11, 13, 5, 6], chaque étape montre les valeurs des variables i, j, et l'état du tableau après chaque itération de la boucle interne.

arr = [12, 11, 13, 5, 6]

n = 5

- 1ère itération externe (i=0):
 - j=0:
arr[0] > arr[1] (True)
arr = [11, 12, 13, 5, 6]
 - j=1:
arr[1] > arr[2] (False)
Aucun échange
 - j=2:
arr[2] > arr[3] (True)
arr = [11, 12, 5, 13, 6]

- j=3:
 - arr[3] > arr[4] (True)
 - arr = [11, 12, 5, 6, 13]
- 2ème itération externe (i=1):
 - j=0:
 - arr[0] > arr[1] (False)
 - Aucun échange
 - j=1:
 - arr[1] > arr[2] (True)
 - arr = [11, 5, 12, 6, 13]
 - j=2:
 - arr[2] > arr[3] (True)
 - arr = [11, 5, 6, 12, 13]
- 3ème itération externe (i=2):
 - j=0:
 - arr[0] > arr[1] (True)
 - arr = [5, 11, 6, 12, 13]
 - j=1:
 - arr[1] > arr[2] (True)
 - arr = [5, 6, 11, 12, 13]
- 4ème itération externe (i=3):
 - j=0:
 - arr[0] > arr[1] (False)
 - Aucun échange
- 5ème itération externe (i=4):

Return [5, 6, 11, 12, 13]

Conclusion :

Le tableau final trié est : **[5, 6, 11, 12, 13]**

Tri par insertion :

L'algorithme de tri par insertion :

```
def tri_insertion(arr):  
    for i in range(1, len(arr)):  
        j = i  
  
        while arr[j-1] > arr[j] and j > 0:  
            arr[j-1], arr[j] = arr[j], arr[j-1]  
            j -= 1  
  
    return arr  
  
a = [22, 31, 13, 15, 1]  
a_trié = tri_insertion(a)  
print(a_trié)  
  
[1, 13, 15, 22, 31]
```

Le Suivi de l'algorithme :

Pour le tableau [22, 31, 13, 15, 1], chaque étape montre les valeurs des variables i, j, et l'état du tableau après chaque itération de la boucle interne.

arr = [22, 31, 13, 15, 1]

- 1ère itération externe (i=1):
 - j=1:
arr[0] > arr[1] and 1 > 0 (False)
Aucun échange nécessaire puisque arr[0] <= arr[1].
- 2ème itération externe (i=2):
 - j=2:
arr[1] > arr[2] and 2 > 0 (True)
arr = [22, 13, 31, 15, 1]
 - j=1:
arr[0] > arr[1] and 1 > 0 (True)
arr = [13, 22, 31, 15, 1]

- j=0:
 $\text{arr}[-1] > \text{arr}[0]$ and $0 > 0$ (False)
 Aucun échange nécessaire puisque $j = 0$.
- 3ème itération externe (i=3):
 - j=3:
 $\text{arr}[2] > \text{arr}[3]$ and $3 > 0$ (True)
 $\text{arr} = [13, 22, 15, 31, 1]$
 - j=2:
 $\text{arr}[1] > \text{arr}[2]$ and $2 > 0$ (True)
 $\text{arr} = [13, 15, 22, 31, 1]$
 - j=1:
 $\text{arr}[0] > \text{arr}[1]$ and $1 > 0$ (False)
 Aucun échange nécessaire puisque $\text{arr}[0] \leq \text{arr}[1]$.
- 4ème itération externe (i=4):
 - j=4:
 $\text{arr}[3] > \text{arr}[4]$ and $4 > 0$ (True)
 $\text{arr} = [13, 15, 22, 1, 31]$
 - j=3:
 $\text{arr}[2] > \text{arr}[3]$ and $3 > 0$ (True)
 $\text{arr} = [13, 15, 1, 22, 31]$
 - j=2:
 $\text{arr}[1] > \text{arr}[2]$ and $2 > 0$ (True)
 $\text{arr} = [13, 1, 15, 22, 31]$
 - j=1:
 $\text{arr}[0] > \text{arr}[1]$ and $1 > 0$ (True)
 $\text{arr} = [1, 13, 15, 22, 31]$
 - j=0:
 $\text{arr}[-1] > \text{arr}[0]$ and $0 > 0$ (False)
 Aucun échange nécessaire puisque $j = 0$.

Return [1, 13, 15, 22, 31]

Conclusion :

Le tableau final trié est : **[1, 13, 15, 22, 31]**

Tri par sélection :

L'algorithme de tri par sélection :

```
def tri_selection(arr):
    n = len(arr)

    for i in range(n - 1):
        # Trouver l'élément minimum dans la partie non triée du tableau
        indice_min = i
        for j in range(i + 1, n):
            if arr[j] < arr[indice_min]:
                indice_min = j

        # Échanger l'élément minimum trouvé avec le premier élément
        arr[i], arr[indice_min] = arr[indice_min], arr[i]

    return arr

a = [10, 7, 17, 25, 2]
a_trié = tri_selection(a)
print(a_trié)

[2, 7, 10, 17, 25]
```

Le Suivi de l'algorithme :

Pour le tableau [10, 7, 17, 25, 2], chaque étape montre les valeurs des variables i, j, indice_min et l'état du tableau après chaque itération de la boucle interne.

arr = [10, 7, 17, 25, 2]

- 1ère itération externe (i=0):
 - j=1: arr[1] < arr[0] (True),
indice_min = 1
 - j=2: arr[2] < arr[1] (False)
indice_min = 1
 - j=3: arr[3] < arr[1] (False),
indice_min = 1
 - j=4: arr[4] < arr[1] (True)
indice_min = 4
 - Échanger arr[0] avec arr[4]
arr = [2, 7, 17, 25, 10]
- 2ème itération externe (i=1):

- j=2: arr[2] < arr[1] (False)
indice_min = 1
- j=3: arr[3] < arr[1] (False)
indice_min = 1
- j=4: arr[4] < arr[1] (False)
indice_min = 1
- Échanger arr[1] avec arr[indice_min]
arr = [2, 7, 17, 25, 10]
- 3ème itération externe (i=2):
 - j=3: arr[3] < arr[2] (False)
indice_min = 2
 - j=4: arr[4] < arr[2] (True)
indice_min = 4
 - Échanger arr[2] avec arr[indice_min]
arr = [2, 7, 10, 25, 17]
- 4ème itération externe (i=3):
 - j=4: arr[4] < arr[3] (True)
indice_min = 4
 - Échanger arr[3] avec arr[indice_min]
arr = [2, 7, 10, 17, 25]

Return [2, 7, 10, 17, 25]

Conclusion :

Le tableau final trié est : **[2, 7, 10, 17, 25]**

Exercice 02 :

L'algorithme de la recherche binaire (dichotomique) :

```
def recherche_binaire(arr, cible):  
    bas, haut = 0, len(arr) - 1  
    while bas <= haut:  
        milieu = (bas + haut) // 2  
        if cible == arr[milieu]:  
            return milieu # Indice de l'élément cible  
        elif cible > arr[milieu]:  
            bas = milieu + 1  
        else:  
            haut = milieu - 1 ## cible < arr[milieu]  
    return -1 # Cible non trouvée
```

```
a = [3, 7, 9, 10, 12, 16, 20, 32]  
res = recherche_binaire(a, 20)  
res
```

6

1. Quel est la condition pour que cet algorithme fonctionne bien ?

Pour que cet algorithme fonctionne correctement, il est impératif que le tableau donné en entrée soit trié. La recherche binaire repose en effet sur le principe que le tableau est déjà ordonné.

2. Pour chaque itération, donnez les valeurs des variables « bas » et « haut ».

Pour le tableau [3, 7, 9, 10, 12, 16, 20, 32] avec la cible 20. Chaque étape montre les valeurs des variables bas, haut, milieu, et l'état de l'élément au milieu après chaque itération de la boucle.

Le Suivi de l'algorithme :

arr = [3, 7, 9, 10, 12, 16, 20, 32]

cible = 20

bas = 0

haut = 7

- 1ère itération:
 - bas <= haut (True)
 - milieu = $(0 + 7) // 2 = 3$
 - arr[milieu] = 10
 - Comparaison : cible > arr[milieu] (True)
 - bas = milieu + 1 = 4
- 2ème itération:
 - bas <= haut (True)
 - milieu = $(4 + 7) // 2 = 5$
 - arr[milieu] = 16
 - Comparaison : cible > arr[milieu] (True)
 - bas = milieu + 1 = 6
- 3ème itération:
 - bas <= haut (True)
 - milieu = $(6 + 7) // 2 = 6$
 - arr[milieu] = 20
 - Comparaison : cible == arr[milieu] (True)
 - Return 6

Conclusion :

La cible 20 a été trouvée à l'indice 6 lors de l'itération 3.